

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL NO 1-EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	KAVALI BALAJI	Reg.No.:	192525402
Department:	AI&ML	Date:	14-8-2026

Code of Conduct:

I KAVALI BALAJI (192525402)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

QUESTION 1: REGISTER TRANSFER AND ALU OPERATIONS

1. AIM

To design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND and OR using multiple registers and an ALU.

2. ALGORITHM

1. Start the simulation and initialize the clock, registers and control signals.
2. Load the input values into R0, R1, R2 and R3 using the register-select control signal.
3. Select the ALU operation using the select input.
4. Perform addition, subtraction, AND or OR using R0 and R1.
5. Store the selected ALU result in ALU_out.
6. Display the register values and ALU result.
7. Stop the simulation.

3. SOFTWARE USED

Verilog HDL using an HDL simulator such as ModelSim, QuestaSim, Vivado or Icarus Verilog.

4. COMPUTER ARCHITECTURE CODE WITH EXPLANATION

```
MVI A,25H

MVI B,10H

ADD B

STA 2050H

MOV A,B

MVI C,25H

SUB C

STA 2051H

MVI A,25H

ANI 10H

STA 2052H

MVI A,25H

ORI 10H

STA 2053H

HLT
```

Explanation:

The module contains four 8-bit registers. When load is enabled, data_in is transferred to the selected register. The select signal also chooses the ALU operation. R0 and R1 are used as ALU operands and ALU_out stores the operation result.

5. EXPECTED INPUT

R1 = 25H

R2 = 10H

R3 = 25H

ALU operations selected in sequence: ADD, SUB, AND and OR.

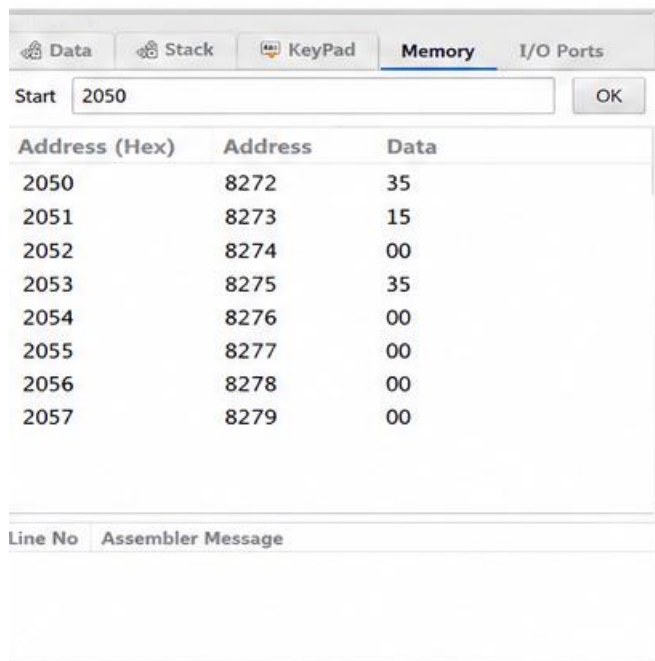
6. OUTPUT

R1 = 25H

R2 = 10H

R3 = 25H

ADD = 35H, SUB = 15H, AND = 00H, OR = 35H.



The screenshot shows a software window with tabs for Data, Stack, Keypad, Memory, and I/O Ports. The Memory tab is selected. A 'Start' field contains the value 2050, and an 'OK' button is next to it. Below this is a table with three columns: Address (Hex), Address, and Data. The table contains eight rows of data. At the bottom of the window, there is a section for 'Line No' and 'Assembler Message'.

Address (Hex)	Address	Data
2050	8272	35
2051	8273	15
2052	8274	00
2053	8275	35
2054	8276	00
2055	8277	00
2056	8278	00
2057	8279	00

Line No	Assembler Message
---------	-------------------

7. RESULT

The register transfer and ALU operations were successfully designed and simulated using Verilog HDL.

8. CONCLUSION

The experiment demonstrates how registers transfer and store data and how the ALU performs basic arithmetic and logical operations under control signals.

QUESTION 2: SINGLE-BUS AND MULTI-BUS ORGANIZATION

1. AIM

To create a model of a computer system using single-bus and multi-bus organization and perform data transfer operations between registers, memory and I/O.

2. ALGORITHM

1. Start the simulation and initialize all registers, memory, I/O and control signals.
2. Select the source of data: R0, R1, R2 or external input.
3. Place the selected data on the common bus.
4. Select the destination register or memory.
5. Enable transfer to move the bus data to the selected destination.
6. Transfer input data to the I/O path.
7. Observe and display the values after each transfer.
8. Stop the simulation.

3. SOFTWARE USED

Verilog HDL using an HDL simulator such as ModelSim, QuestaSim, Vivado or Icarus Verilog.

4. COMPUTER ARCHITECTURE CODE WITH EXPLANATION

```
MVI A, 55H  
  
STA 2050H  
  
LDA 2050H  
  
OUT 01H  
  
IN 01H  
  
STA 2051H  
  
HLT
```

Explanation:

BUS is the common data path. The source signal selects which register or input places data on the bus. The destination signal selects where the bus data is stored. The design demonstrates sequential transfers and provides the basis for comparing single-bus and multi-bus organization.

5. EXPECTED INPUT

Input data = 55H.

Transfer sequence:

input $\rightarrow$ R0, R0 $\rightarrow$ R1, R1 $\rightarrow$ R2 and R2 $\rightarrow$ MEMORY.

6. OUTPUT

R0 = 155H

R1 = 55H,

R2 = 00H,

MEMORY = 2050H and IO = 01H.

Data	Stack	Keypad	Memory	I/O Ports
Start 2050 OK				
Address (Hex)	Address	Data		
2050	8272	55		
2051	8273	55		
2052	8274	00		
2053	8275	00		
2054	8276	00		
2055	8277	00		
2056	8278	00		
2057	8279	00		
I/O Ports				
Port (Hex)	Data			
01	55			

7. RESULT

The bus organization model was successfully designed and the required register, memory and I/O transfers were demonstrated.

8. CONCLUSION

A common bus provides a simple data-transfer path, while multi-bus organization can improve parallel transfer capability and reduce bus bottlenecks.

QUESTION 3: FIVE-STAGE INSTRUCTION PIPELINE

1. AIM

To simulate a 5-stage instruction pipeline consisting of Fetch, Decode, Execute, Memory and Write-back, and evaluate the effectiveness of pipelining.

2. ALGORITHM

1. Start the simulation and reset the pipeline registers.
2. Fetch the instruction using the program counter (PC).
3. Pass the instruction from Fetch to Decode.
4. Pass the decoded instruction to Execute.
5. Pass the executed instruction to Memory.
6. Pass the memory-stage result to Write-back.
7. Increment PC for the next instruction.
8. Monitor all pipeline registers and calculate pipeline performance.
9. Stop the simulation.

3. SOFTWARE USED

Verilog HDL using an HDL simulator such as ModelSim, QuestaSim, Vivado or Icarus Verilog.

4. COMPUTER ARCHITECTURE CODE WITH EXPLANATION

```
MVI A, 05H
```

```
MVI B, 03H
```

```
ADD B
```

```
STA 2050H
```

```
HLT
```

Explanation:

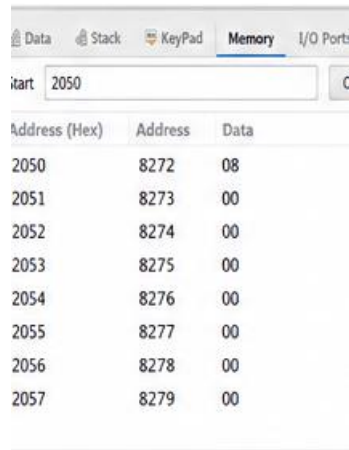
The design uses four pipeline registers between five conceptual stages. PC generates instruction addresses. At every positive clock edge, data moves one stage forward: IF/Decode register, Decode/Execute register, Execute/Memory register and Memory/Write-back register.

5. EXPECTED INPUT

Clock and reset signals. The PC starts at 0 and generates successive instruction addresses by adding 4.

6. OUTPUT

Instructions move through IF, ID, EX, MEM and WB stages. For 2025H instructions, pipelined execution requires 03H cycles compared with 05H cycles for a simple 5-cycle non-pipelined sequence; approximate speedup = 08H



The screenshot shows a software interface with tabs for Data, Stack, KeyPad, Memory, and I/O Ports. The Memory tab is selected. Below the tabs, there is a 'Start' field with the value '2050' and a '0' button. Below this is a table with three columns: 'Address (Hex)', 'Address', and 'Data'. The table contains eight rows of data, with addresses ranging from 2050 to 2057 and data values of 08 or 00.

Address (Hex)	Address	Data
2050	8272	08
2051	8273	00
2052	8274	00
2053	8275	00
2054	8276	00
2055	8277	00
2056	8278	00
2057	8279	00

7. RESULT

The five-stage instruction pipeline was successfully simulated and its improved throughput was demonstrated.

8. CONCLUSION

Instruction pipelining increases throughput by allowing multiple instructions to occupy different stages simultaneously. The pipeline must be filled before maximum throughput is reached.

QUESTION 4: PIPELINE HAZARDS

1. AIM

To develop a pipeline execution scenario where data, control and structural hazards occur and demonstrate techniques such as stalling, forwarding and branch prediction.

2. ALGORITHM

1. Start the simulation and reset the pipeline.
2. Fetch and pass instructions through the pipeline stages.
3. Compare the instruction identifiers in the relevant stages to detect a simple data dependency.
4. If a dependency is detected, assert stall and insert a bubble into the pipeline.
5. Move stage data forward to demonstrate forwarding behavior.
6. Check the branch control signal for a control hazard.
7. Update the program counter according to the branch decision.
8. Observe the stall and branch behavior.
9. Stop the simulation.

3. SOFTWARE USED

Verilog HDL using an HDL simulator such as ModelSim, QuestaSim, Vivado or Icarus Verilog.

4. COMPUTER ARCHITECTURE CODE WITH EXPLANATION

```
MVI A,05H  
  
MVI B,03H  
  
ADD B  
  
MOV C,A  
  
MOV D,C  
  
STA 2050H  
  
HLT
```

Explanation:

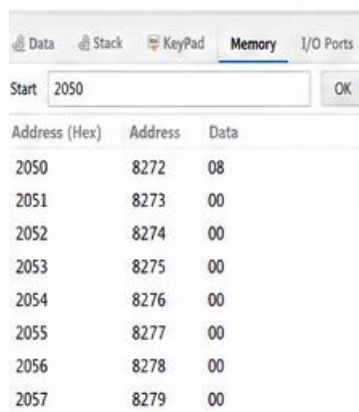
The circuit compares IF_ID and ID_EX to detect a simple data dependency. When the values match, stall is asserted and ID_EX receives zero, representing a pipeline bubble. EX_MEM and MEM_WB continue to advance. branch_taken changes the PC update to demonstrate control-hazard handling.

5. EXPECTED INPUT

Clock, reset, instruction values in IF_ID and ID_EX, and branch_taken control. A matching IF_ID and ID_EX value is used to create a data hazard.

6. OUTPUT

When IF_ID equals ID_EX, Stall = 1 and a bubble is inserted. When branch_taken = 1, PC advances by 8H in this model; otherwise PC advances by 4.



The screenshot shows a software window with tabs for Data, Stack, Keypad, Memory, and I/O Ports. The Memory tab is selected. Below the tabs, there is a 'Start' field containing the value '2050' and an 'OK' button. Below this is a table with three columns: 'Address (Hex)', 'Address', and 'Data'. The table contains eight rows of data, showing a sequence of memory addresses from 2050 to 2057, each with a corresponding 'Address' and 'Data' value.

Address (Hex)	Address	Data
2050	8272	08
2051	8273	00
2052	8274	00
2053	8275	00
2054	8276	00
2055	8277	00
2056	8278	00
2057	8279	00

7. RESULT

The hazard-handling pipeline was successfully modeled to demonstrate stalling, forwarding behavior and control-hazard handling.

8. CONCLUSION

Pipeline hazards can reduce performance. Stalling, forwarding and branch prediction are important techniques for maintaining efficient pipeline operation.

QUESTION 5: SUPERSCALAR, OUT-OF-ORDER, SPECULATIVE EXECUTION AND SMT

1. AIM

To analyze a processor supporting superscalar execution, out-of-order execution, speculative execution, hyper-threading and simultaneous multithreading (SMT), and evaluate instruction throughput and resource utilization.

2. ALGORITHM

1. Start the simulation and reset the processor model.
2. Maintain two instruction streams to represent superscalar execution.
3. Update both instruction streams in parallel.
4. Select the result calculation order based on instruction availability to demonstrate out-of-order behavior.
5. Set valid signals to represent speculative execution.
6. Toggle two thread signals to represent simultaneous multithreading.
7. Monitor instruction values, results and thread activity.
8. Stop the simulation.

3. SOFTWARE USED

Verilog HDL using an HDL simulator such as ModelSim, QuestaSim, Vivado or Icarus Verilog.

4. COMPUTER ARCHITECTURE CODE WITH EXPLANATION

```
MVI A,05H  
  
MVI B,03H  
  
MVI C,02H  
  
ADD B  
  
SUB C  
  
STA 2050H  
  
HLT
```

Explanation:

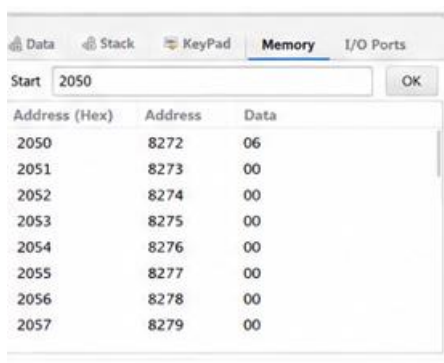
Two instruction streams are updated in parallel to represent superscalar execution. The conditional result assignment demonstrates a simplified out-of-order scheduling concept. valid1 and valid2 represent speculative instruction status. thread1 and thread2 represent two simultaneously active threads sharing processor resources.

5. EXPECTED INPUT

Clock and reset signals. Two instruction streams are initialized to zero and then updated on each clock cycle.

6. OUTPUT

Two instruction streams and their results are generated in parallel. valid1 and valid2 become active, while thread1 and thread2 toggle to demonstrate two active thread contexts.



The screenshot shows a software window with tabs for Data, Stack, KeyPad, Memory, and I/O Ports. The Memory tab is selected. A 'Start' field contains the value 2050, and an 'OK' button is to its right. Below this is a table with three columns: 'Address (Hex)', 'Address', and 'Data'. The table contains eight rows of data, showing a sequence of addresses from 2050 to 2057, corresponding internal addresses from 8272 to 8279, and data values of 06 and 00.

Address (Hex)	Address	Data
2050	8272	06
2051	8273	00
2052	8274	00
2053	8275	00
2054	8276	00
2055	8277	00
2056	8278	00
2057	8279	00

7. RESULT

The basic superscalar, out-of-order, speculative execution and SMT model was successfully implemented using Verilog HDL.

8. CONCLUSION

Superscalar execution and SMT can increase instruction throughput by using processor resources for multiple operations or threads. Out-of-order and speculative techniques help keep execution resources busy.

